

נושאים: מחלקות ואובייקטים, הורשה, פולימורפיזם ודפוסי עיצוב

שם הסטודנט/ית:	תאריך: _____
משך הבחינה: 90 דקות	חומר עזר: ללא חומר עזר, אלא אם המרצה אישר/ה אחרת
סה"כ: 100 נקודות	יש לענות בכתב ברור ולנמק בשאלות הפתוחות
הנחיה חשובה	בשאלות הקוד יש להתייחס לשפת Java / OOP כללית, אלא אם מצוין אחרת

חלק א' - שאלות אמריקאיות (32 נקודות)

בחרו תשובה אחת נכונה בכל שאלה. כל שאלה שווה 4 נקודות.

1. איזה משפט מתאר בצורה המדויקת ביותר את ההבדל בין מחלקה לאובייקט?

א. מחלקה היא מופע שנוצר בזמן ריצה, ואובייקט הוא תבנית.

ב. מחלקה היא תבנית שמגדירה שדות ומתודות, ואובייקט הוא מופע שלה בזיכרון.

ג. מחלקה יכולה להכיל רק נתונים, ואובייקט יכול להכיל רק פעולות.

ד. אין הבדל מעשי ביניהם בתכנות מונחה עצמים.

2. מהי המטרה העיקרית של בנאי (**Constructor**)?

א. להרוס אובייקט בסוף השימוש.

ב. לאתחל את מצב האובייקט בזמן יצירתו.

ג. להפעיל פולימורפיזם בין מחלקות לא קשורות.

ד. להגדיר מחלקה כ-abstract.

3. איזו אפשרות ממחישה הכמסה (**Encapsulation**)?

כדי שיהיו נגשים מבחן public-א. הגדרת כל השדות כ

ב. הסתרת שדות פנימיים ומתן גישה מבוקרת דרך מתודות.

ג. יצירת כמה מחלקות שיורשות מאותה מחלקת אב.

ד. שימוש באותו שם מתודה עם מימוש שונה במחלקות יורשות בלבד.

4. במחלקה שיורשת ממחלקת אב, מה מאפשרת המילה **super** בדרך כלל?

א. גישה לבנאי או למתודות של מחלקת האב.

ב. הפיכת אובייקט למחלקה סטטית.

ג. מניעת דריסה של מתודות.

ד. יצירת ממשק אוטומטי למחלקה.

5. איזה מצב הוא דוגמה לפולימורפיזם?

א. משתנה מסוג Animal מפנה לאובייקט מסוג Dog והקריאה speak() מפעילה את המימוש של Dog.

ב. מחלקה אחת מחזיקה שני שדות פרטיים.

ג. שתי מחלקות נמצאות באותו קובץ.

ד. אובייקט נוצר ללא בנאי מפורש.

6. איזה דפוס עיצוב מתאים כאשר רוצים להפריד בין יצירת אובייקטים לבין הקוד שמשתמש בהם?

א. Singleton

מבחן תכנות מונחה עצמים

ב. Factory Method

ג. Observer

ד. Adapter

7. איזה דפוס עיצוב מתאים למצב שבו כמה אובייקטים צריכים להתעדכן כאשר אובייקט מרכזי משנה מצב?

א. Observer

ב. Builder

ג. Singleton

ד. Strategy

8. איזו טענה נכונה לגבי ממשק (**Interface**)?

א. ממשק מחייב מחלקות מממשות לספק מימוש לפעולות המוגדרות בו.

ב. ממשק תמיד מכיל שדות פרטיים בלבד.

ג. אי אפשר להחזיק משתנה מטיפוס של ממשק.

ד. ממשק נועד רק ליצירת אובייקטים בודדים במערכת.

חלק ב' - נכון / לא נכון (16 נקודות)

סמנו: נכון או לא נכון. כל שאלה שווה 2 נקודות.

1. מחלקה **abstract** יכולה להכיל גם מתודות ממומשות וגם מתודות ללא מימוש.

סמנו: נכון / לא נכון

2. אם שדה מוגדר **private**, אף מתודה בתוך אותה מחלקה לא יכולה לגשת אליו.

סמנו: נכון / לא נכון

3. **Overriding** מתרחש כאשר מחלקה יורשת מספקת מימוש חדש למתודה שהוגדרה במחלקת האב.

סמנו: נכון / לא נכון

4. **Overloading** הוא מצב שבו שתי מתודות בעלות אותו שם חייבות להחזיר אותו טיפוס בדיוק.

סמנו: נכון / לא נכון

5. פולימורפיזם מאפשר לכתוב קוד כללי יותר שתלוי בטיפוס אב או בממשק.

סמנו: נכון / לא נכון

6. דפוס **Singleton** מבטיח בדרך כלל שיהיה מופע יחיד למחלקה מסוימת.

סמנו: נכון / לא נכון

7. בקוד הלקוח **new** מחייב שכל האובייקטים ייווצרו ישירות עם **Factory** דפוס.

סמנו: נכון / לא נכון

8. בהורשה, מחלקת בן מקבלת אוטומטית גישה ישירה לכל השדות הפרטיים של מחלקת האב.

סמנו: נכון / לא נכון

חלק ג' - שאלות קוד וניתוח (30 נקודות)

ענו בקצרה אך נמקו. בשאלות פלט כתבו את הפלט המדויק או הסבירו מדוע יש שגיאה.

1. מה יודפס בקוד הבא? הסבירו מה קורה מבחינת הפניות לאובייקטים. (6 נקודות)

```
class Counter {  
    private int value;  
  
    public Counter(int value) {  
        this.value = value;  
    }  
}
```

```

    public void inc() {
        value++;
    }

    public int getValue() {
        return value;
    }
}

Counter c1 = new Counter(3);
Counter c2 = c1;
c2.inc();
System.out.println(c1.getValue());

```

תשובה:

2. מה יודפס ומדוע? ציינו איזה עיקרון **OOP** בא לידי ביטוי. (6 נקודות)

```

class A {
    public void print() {
        System.out.println("A");
    }
}

class B extends A {
    public void print() {
        System.out.println("B");
    }
}

A x = new B();
x.print();

```

תשובה:

3. אם ניצור **Car** ונפעיל **show()**, מה יודפס בשתי השורות? הסבירו את משמעות **super**. (6 נקודות)

```

class Vehicle {
    protected int speed = 10;
}

class Car extends Vehicle {
    private int speed = 30;

    public void show() {
        System.out.println(speed);
        System.out.println(super.speed);
    }
}

```

תשובה:

4. כתבו קטע קוד קצר שמחזיק מערך או רשימה של **Payable** ומחשב סכום תשלומים כולל. איזה יתרון יש לשימוש בממשק כאן? (6 נקודות)

```

interface Payable {
    double pay();
}

```

מבחן תכנות מונחה עצמים

```

class Employee implements Payable {
    public double pay() { return 5000; }
}

class Freelancer implements Payable {
    public double pay() { return 1200; }
}

```

תשובה: _____

5. איזה דפוס עיצוב מופיע בקוד? ציינו יתרון אחד וסיכון/חיסרון אחד בשימוש בו. (6 נקודות)

```

class Logger {
    private static Logger instance;

    private Logger() {}

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
}

```

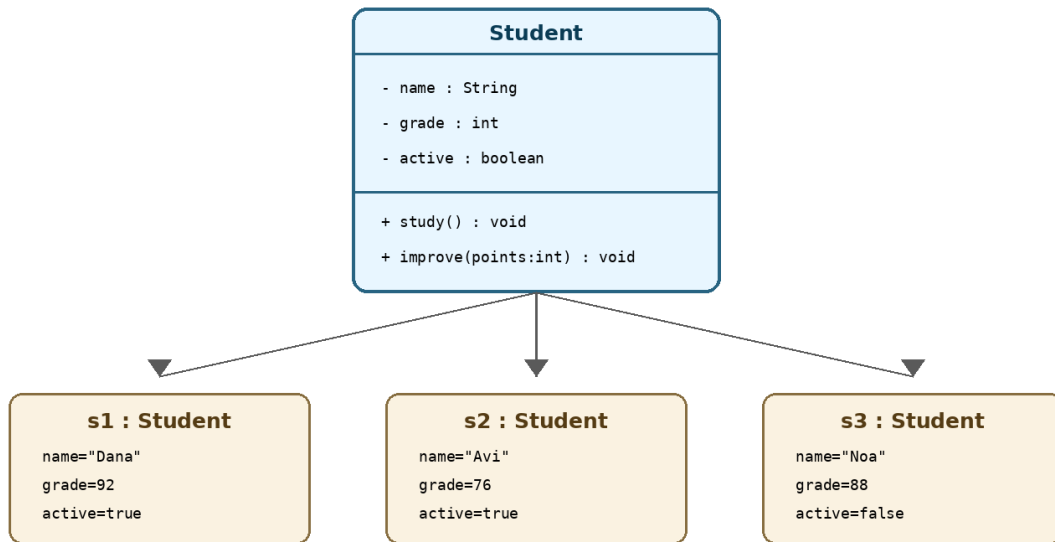
תשובה: _____

חלק ד' - שאלות ויזואליות / איורים (22 נקודות)

התבוננו באיורים וענו. המטרה היא לזהות רעיונות OOP מתוך תרשים ולא רק מתוך קוד.

1. באיזה חלק באיור מוצגת המחלקה ובאיזה חלק מוצגים האובייקטים? כתבו גם שני הבדלים בין מחלקה לאובייקט. (7 נקודות)

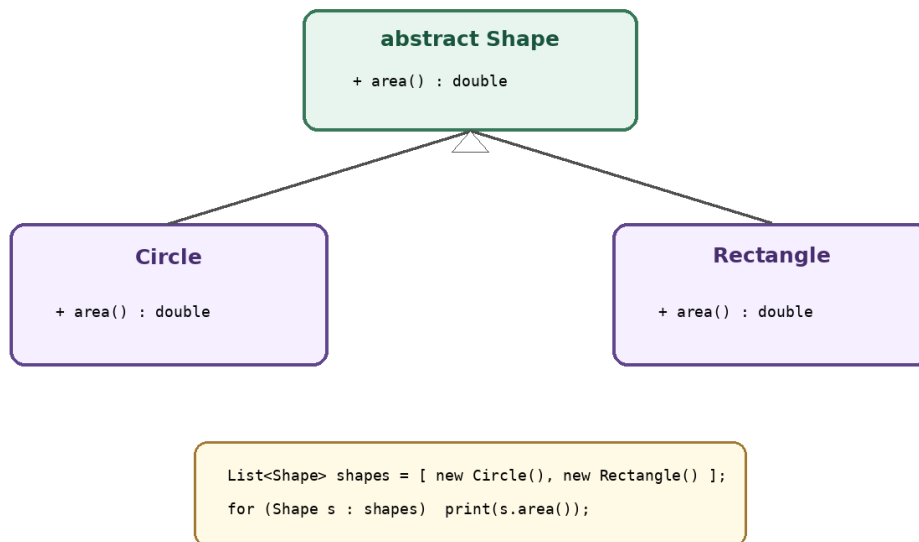
איור 1 - מחלקה ואובייקטים



תשובה:

2. באיזו תכונה של **OOP** עוסק האיור? הסבירו כיצד הקריאה **area()** יכולה להפעיל מימוש שונה עבור כל אובייקט. (7 נקודות)

איור 2 - קשרי הורשה ופעולה פולימורפית

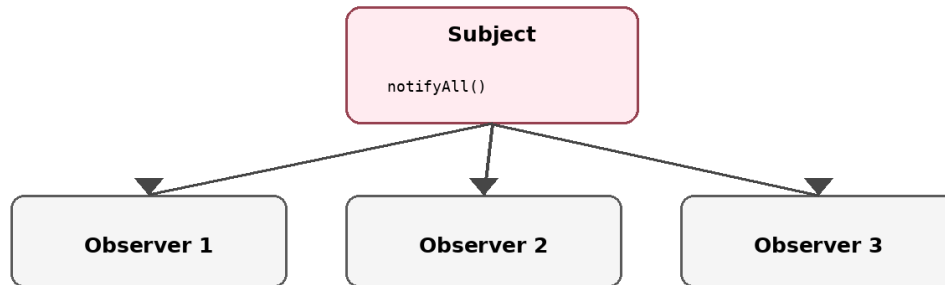
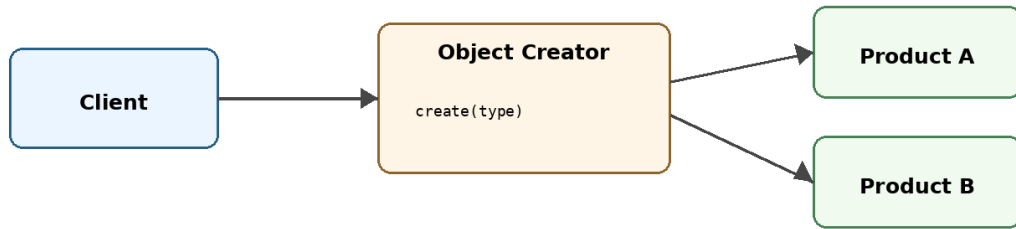


תשובה:

3. באיור מופיעים שני רעיונות שונים מדפוסי עיצוב. זהו את הדפוסים האפשריים והסבירו בקצרה מה תפקיד כל אחד מהם. (8 נקודות)

מבחן תכנות מונחה עצמים

איור 3 - תרחיש מערכת עם יצירה ועדכון



תשובה:

נספח תשובות מקוצר - לבדיקה עצמית

מומלץ לפתור קודם ורק אחר כך לבדוק. בחלקים הפתוחים ייתכנו ניסוחים נכונים נוספים.

חלק א'

1-ב; 2-ב; 3-ב; 4-א; 5-א; 6-ב; 7-א; 8-א

חלק ב'

1 נכון; 2 לא נכון; 3 נכון; 4 לא נכון; 5 נכון; 6 נכון; 7 לא נכון; 8 לא נכון

חלק ג' - עיקרי תשובה

1. יודפס 4, כי c1 ו-c2 מפנים לאותו אובייקט בזיכרון.

2. יודפס B. זה פולימורפיזם / Dynamic Dispatch: הטיפוס בפועל הוא B.

3. יודפס 30 ואז 10. speed הראשון הוא השדה של Car, ו-super.speed מתייחס לשדה במחלקת האב Vehicle.

4. אפשר ליצור Payable[] ולסכם p.pay() בלולאה. היתרון: קוד גמיש שעובד מול ממשק ולא מול מחלקה מסוימת.

5. Singleton. יתרון: גישה למופע יחיד ומשותף; חיסרון: תלות גלובלית, קושי בבדיקות, ולעיתים בעיות Thread Safety.

חלק ד' - עיקרי תשובה

1. המלבן Student הוא המחלקה; הכרטיסים s1, s2, s3 הם אובייקטים. מחלקה היא תבנית, אובייקט הוא מופע עם ערכים משלו.

מבחן תכנות מונחה עצמים

2. הורשה ופולימורפיזם. משתנה מסוג Shape יכול להחזיק Circle או Rectangle, והקריאה area() מפעילה את המימוש של הטיפוס בפועל.
3. בחלק העליון: Factory Method / Factory - הפרדת יצירת אובייקטים מהלקוח. בחלק התחתון: Observer - אובייקט מרכזי מודיע למאזינים על שינוי.